PROJECT TECHNICAL DOCUMENTATION

BioSpecInfo

Web platform for computational chemistry, physics and mathematics

AI Agent Architecture "Spectra"

DOCUMENT CONTROL

Identifier	`BSI-DOC-05-EN`
Title	AI Agent Architecture "Spectra"
Software version	**bsi-v251**
Revision (commit)	`ddd84ed`
Date of issue	2026-11-12
Author	Samuele Pio Provenzano
Language	English
Status	Issued with version bsi-v251

Document generated from the versioned source in the project repository. The version and revision shown here and in the footer of every page identify exactly the state of the software described.

Contents

1. Executive summary

2. Architecture

3. The 51 tools

4. Notable engineering decisions

5. Verification

6. Quantitative summary

7. Stated limitations

AI Agent Architecture "Spectra" — BioSpecInfo

Field	Value
Project	BioSpecInfo — Spectra component (agentic AI copilot)
Author	Samuele Pio Provenzano
Thesis supervisor	Prof. Savino Longo — University of Bari Aldo Moro
Component	`bsi-ai-hub.js` — 6,254 lines, zero runtime dependencies
Type	Multi-provider conversational agent with client-side tool execution
Repository	`samupropio1-ship-it/BioSpecInfo-v11`
Documented version	Service Worker `bsi-v251`

1. Executive summary

Spectra is an **agent**, not a thin wrapper around a language model. The difference is measurable: the model does not answer quantitative questions from memory — it **invokes deterministic tools** that compute the result, and the control loop lets it **chain up to 10 calls**, verifying each step before answering.

The architecture addresses three problems that separate a usable agent from a demo:

1. **Grounding** — a fabricated chemical value can be dangerous. 51 tools cover

computation, public databases and the application's own datasets; the system prompt explicitly forbids quoting numeric values from memory.

1. **Transparency** — the model's reasoning is streamed live and stored

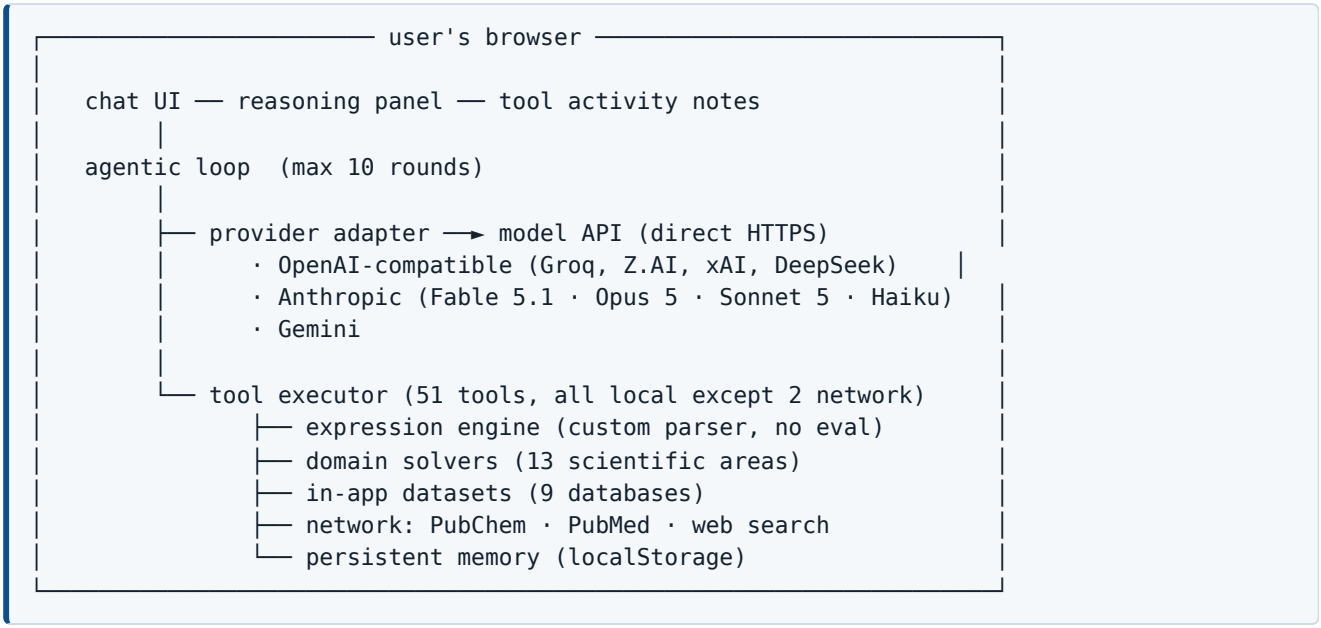
alongside the answer, together with the list of tools actually invoked. The agent's work is auditable after the fact.

1. **Provider portability** — a single tool registry is translated into the

three function-calling formats in use today (OpenAI-compatible, Anthropic, Gemini), so the agent behaves identically across ten model configurations, four of them free.

2. Architecture

2.1 Overview



There is no backend. API keys stay in the browser's `localStorage` and travel only to the selected provider: no data passes through BioSpecInfo servers, which do not exist.

2.2 Agentic loop

Each round: send the conversation history (last 40 turns) plus the system prompt and the schema of all 51 tools → receive the streamed response → if it contains tool calls, execute **all** of them → rebuild the assistant turn in the provider's native format → repeat.

Three exit conditions: no tool calls (final answer), `MAX_ROUNDS = 10` exhausted, or user interruption.

2.3 Provider-neutral tool registry

Tools are declared once in JSON Schema and translated at runtime:

Family	Required shape
Anthropic	`{name, description, input_schema}`
Gemini	`[{functionDeclarations: [...]}]`
OpenAI-compatible	`{type:"function", function:{...}}`

The same applies to messages: turns containing tool calls are serialised into each provider's native form and kept in a `_native` field, so a conversation stays coherent even when switching models.

2.4 No hard-coded model names

A model name written into the code is a time bomb: it works until the provider retires that model, and then the application stops with a 404 the user cannot fix. It has happened twice in this project:

Provider	Error	When
Google	`models/gemini-1.5-flash is not found for API version v1beta`	removed from `v1beta`
Groq	`The model llama-3.3-70b-versatile does not exist or you do not have access to it`	deprecated 17/06/2026

On some services the problem is structural: free model ids change constantly, by design. (That is why OpenRouter was later dropped from the list, see 2.4.)

The fix is not to write the new name — that would be the same bomb with a longer fuse — but to **remove the name** and ask the API, which knows what exists *for that key*. This also resolves the ambiguity in Groq's message: "does not exist" **or** "you do not have access" are different cases, and a per-key listing tells them apart.

Every configuration with `modelliCandidati` starts at `model: null` and is resolved at runtime. The two families use different strategies because their names are different:

Gemini — version scoring. Names follow a regular scheme (`gemini-<major>.<minor>-<family>`), so they can be ordered: newest version first, `flash` family preferred, with penalties for experimental, `preview` and `-lite` variants; models without `streamGenerateContent` are discarded, as are non-conversational ones (embedding, image, native audio, Live API). When Gemini 3.0 ships it will be picked automatically.

OpenAI family — candidates in preference order. Here names are not comparable to each other (`openai/gpt-oss-120b` versus `qwen/qwen3.6-27b`) and automatic scoring would choose badly. `GET /models` is queried and the **first candidate that still exists** wins: the order of the list *is* the preference, and the listing serves to skip the ones that vanished. Only if no candidate survives does a generic score run over the available models — discarding transcription, speech, embedding and guard models, and preferring the larger model all else being equal.

Three fallback levels in both cases: model listing → candidate probing → first candidate, letting the real error from the generation call speak instead of inventing one.

The choice is cached for seven days per provider, keyed to a 32-bit FNV-1a fingerprint of the API key (different keys see different catalogues; the key itself is never duplicated). If the model is retired *while* cached, the `404` — or a `400` whose text mentions the model, as some providers return — invalidates the cache, re-resolves and retries **once**: a flag prevents an infinite loop when the new model fails too.

Anthropic is the deliberate exception: its models are paid, explicitly chosen by the user, and deprecated with long notice.

The three free services, chosen one by one

Free quality is not all alike, and the difference matters: an 8-billion-parameter model does not hold up on a multi-step physical-chemistry problem. The list has been **shortened**, not extended — two services were removed because they made answers worse, not better.

Service	Models	Its role
Groq	GPT-OSS 120B, Qwen3	The workhorse: 131K context, ~30 requests/minute
Google Gemini	Gemini Flash	Up to 1M context: whole documents and PDFs
Z.AI GLM	GLM-4.7-Flash	Free **with no expiry**: the reserve still there when credits are gone

Removed. *GitHub Models*: it was here, and was removed because GitHub **retired it entirely on 30 July 2026** — playground, catalogue and inference API switched off for everyone. It had been added two days earlier on the strength of pages describing the still-live service: the lesson is that verifying *a thing exists* is not verifying *it is still alive*, and the search to run is "<name> retired / shutdown / deprecated <year>". The failure did not stop at the dropdown entry: the rank assigned to it also made it Core Mode's automatic choice.

Mistral: mid-tier quality and a free plan requiring consent to data training — not worth paying for unpublished thesis material when five alternatives carry no such clause. *OpenRouter*: small free models with ids that change constantly, unsuited to an agent chaining ten tool calls. *Cerebras*: since August 2026 there is no card-free plan, and free context is capped at 8K — Spectra sends 32 tool definitions on top of history, which does not fit.

None of the five matches a paid frontier model on the hardest problems; GitHub Models come closest, at the cost of low request ceilings. That is why the choice stays the user's rather than imposing one service.

The paid frontier

Free tiers are enough for studying; for a hard problem they are not. The selection criterion was **GPQA Diamond** — graduate-level physics, chemistry and biology questions — because it is the only benchmark measuring what this app actually asks.

Service	Why it is here
GPT-5.6	94.6% on GPQA Diamond: the highest score available today. ~$4/M input
Gemini 3 Pro	Over 1M context, and **the same key** as free Gemini. ~$2/M
DeepSeek V4	High-tier reasoning at ~$0.66/M: the best quality/price ratio
Grok 4.6	First on agentic tool calling and lowest hallucination rate — exactly the profile an agent chaining tools needs. ~$2/M
Claude (Fable 5.1, Opus 5, Sonnet 5)	One key for all three; the only ones with the Python sandbox in Core Mode

Two configurations can use **the same key**: the four Claude models are one Anthropic account, and Gemini Flash and Gemini 3 Pro one AI Studio key. `chiaveCondivisaCon` points them at the same entry, so the user pastes it once. Clearing it clears it for all twins — leaving one behind would make the key reappear on the other.

The two Gemini entries forced the scorer to be generalised: they share family, endpoint and key but want **opposite** models. The preference is now declared in configuration (`/flash/` versus `/pro/`) and outweighs the rest of the score. Without it the free configuration would have picked `gemini-3-pro` — a newer generation, therefore a higher score, but **paid**: a model the free key cannot use.

The same pass surfaced a second issue: the minor version in the name is **optional**. From generation 3 Google dropped it (`gemini-3-pro`, not `gemini-3.0-pro`), and the regex requiring the dot scored the *newest* models as generic aliases — so they lost to older ones, and the automatic upgrade described in 2.4 would never have triggered.

The provider audit, and why it must be repeated

The system repairs itself on **model names** — it queries the catalogue and picks among what exists. It does not repair itself against a **dead service**: that must be checked by hand, and GitHub Models' retirement showed what not doing so costs.

September 2026 check, provider by provider:

Provider	Result
Groq	Active. `llama-3.3-70b-versatile` **is no longer served** as of August 2026 and `llama-3.1-8b-instant` is deprecated: removed from candidates, leaving Groq's own replacements
Google Gemini	Active
NVIDIA NIM	Removed 09/2026: alive, but without CORS headers it cannot be called from a web page
Z.AI	Active, GLM-4.7-Flash and 4.5-Flash still free
OpenAI, DeepSeek, Anthropic	Active
xAI	Active, but candidates were **two generations** behind: the top is `grok-4.6` since 12/08/2026

Two traps recorded so they are not repeated. First: never put a paid model among a free configuration's candidates — true for `glm-5.x` on Z.AI as it was for `gemini-3-pro`, or the configuration picks something the free key cannot use. Second: a dead provider with a high rank does not merely break a dropdown entry, it **becomes Core Mode's automatic choice**.

When a provider does not answer

A `fetch` that fails **before** receiving a response means one of three things: no network, the service does not accept direct browser calls (CORS), or the service no longer exists. The browser, for security reasons, does not let you tell which — the error is deliberately generic.

This used to stop the turn. But if the three causes cannot be told apart, the right response is the same for all three: **move to another provider** the user holds a key for, exactly as for exhausted quota. It is the behaviour that would have masked GitHub Models' retirement entirely, instead of leaving every request stuck on a dead service.

Errors that would repeat identically everywhere stay out of the fallback — a wrong key, a malformed request: trying them on every provider and then reporting a random last error would hide the real cause.

Removing a service from the list is a breaking operation, and deserves a note: whoever had it selected has that name saved in `localStorage`, and without validation would hit `PROVIDERS[undefined]` — Spectra would no longer open. `getSavedProvider()` always validates against the registry and falls back to an existing service.

2.5 The optional proxy — Spectra without a key

The API key lives in the user's browser: with no backend there is no alternative, and it is the limitation stated openly at the end of this document. Every visitor has to obtain their own free key and paste it in.

`proxy/spectra-proxy.js` is the way out, and it sits **outside** the page: a Cloudflare Worker (free plan) holding the keys as server-side secrets. When it is configured the browser carries no key at all, and anyone opening BioSpecInfo uses Spectra without entering anything.

It is not a plain forwarder:

- **The path passes through untouched.** The route is `/<provider>/<path>` and

the rest is relayed as received: the proxy knows nothing about API shapes and keeps working when Spectra changes model or parameters.

- **Several keys per provider, with automatic takeover.** Each secret is a

comma-separated list; on `429` (quota) or `401/403` (revoked key) it moves to the next one within the same request. Not on `400` — that is a bad request and would fail identically.

- **Streaming is never buffered**: the response body is passed straight

through, otherwise answers would appear all at once at the end.

- **It is not an open relay.** Credentials arriving from the client

(`Authorization` , `x-api-key` , `?key=`) are discarded and replaced: nobody can use the proxy to push a key of their own.

- **Three brakes** against abuse: an allowed-origins list, a per-IP limit and a

daily cap.

On the Spectra side the graft is in one place. `GET /stato` reports which providers the proxy actually covers, so a model whose secret is missing is never offered as "no key needed"; `buildRequest` routes to the proxy **and omits authentication entirely**; `chiaveDaUsare()` returns a placeholder so the "missing key" guards do not block sending — a placeholder that never leaves the browser.

Both paths coexist: without `PROXY_URL` everything works as before, and for providers the proxy does not cover Spectra keeps using the local key.

2.6 Starting over

Everything Spectra remembers lives in `localStorage`: chats, history, API keys, persistent memory, review cards. A "clear history" button that removed only the conversations would not start over — reopening the app would restore the same state.

The delicate part is that keys exist in **two formats**: the current map (`bsi_api_keys`, one key per provider) and the single-key format of earlier versions (`bsi_api_key`). `getKeysMap()` migrates the latter into the former, so clearing only the map **brings the old key back** on the next access. Both must go — along with the resolved-model caches, which mean nothing without a key.

That is why the list of everything the application writes lives in one place, `DATI_CANCELLABILI`, split into five groups with a label and a description. The confirmation panel is built from it: it shows how many entries actually exist per group, disables the empty ones and requires a second explicit click. Only chats and keys are pre-selected; memory, review scheduling and the rest of the app's progress must be chosen deliberately, because they erase work unrelated to starting the conversation over.

Deliberately excluded: the PRO licence, the trial period and the device identity — data a user does not expect to lose by emptying a chat, and a licence must never be deleted by accident. The panel says so.

After deletion the in-process state is reset too — the list of providers the proxy covers, the resolved model per provider — otherwise it would stay valid until the page reloads, and Spectra would keep using a model chosen with a key that no longer exists.

2.7 Making free keys carry real load

Free tiers impose two different limits, and each needs a different answer.

The per-request ceiling. GitHub Models accepts 8,000 input tokens. Spectra's fixed cost is ~8,150 — 2,009 for the system prompt plus **6,128 for tool definitions alone** — so that service would not even start: it would fail before the user typed a word. The measurement came before the fix, and changed what the fix had to be.

The answer is not to trim history — it is to trim **the fixed cost**. Tools are paid on every turn; the conversation is the content. Of the two, the fixed cost must shrink first. `adattaAlBudget()` selects the tools **relevant** to the question instead of all 32, then shortens history with what remains.

Relevance is scored against a keyword map kept **outside** the tool registry: the words people use to *ask* for something are not the words a tool is *documented* with. The description of `astrofisica` talks about Wien and Stefan-Boltzmann; the user writes "what temperature is this nebula". Matching the question against descriptions alone picked `farmacocinetica` for an astrochemistry question — measured, not assumed. Matching is by stem (first five characters) because Italian inflects: "nebulosa" and "nebulose" must count as the same word.

Rate limits. A `429` on a free tier is almost always the *per-minute* limit, not an exhausted quota: waiting a few seconds clears it. The provider says how long in `Retry-After`, or in the error text ("*Please try again in 7.5s*"); when it says nothing, exponential backoff with a little jitter, so every open tab does not retry at the same instant. If the requested wait is absurd, it does not wait at all: better to say "quota exhausted" than to leave the user watching an hourglass for an hour.

Provider fallback. When a quota really is exhausted, the turn is redone on another service the user holds a key for. This is why several free keys together carry load that none of them carries alone.

Two non-negotiable constraints:

- It restarts from the **original** messages, not from half-finished history.

Turns containing tool calls are stored in the previous provider's native format and cannot be handed to another. The turn's work is lost; a correct answer is gained.

- Fallback uses **free services only**. Silently moving to a paid one would

spend the user's money without their say-so; if the chosen service was paid, it still gets the first attempt.

And it moves on **only** for exhausted quota. A `401` or a malformed request would repeat identically everywhere: trying all providers and then reporting a random last error would hide the real cause.

2.8 Core Mode, and saying what is happening

The mode. One switch that changes four things at once, because none of them alone would move the needle: it switches to the most capable configuration among those with a key, raises agentic rounds from 10 to 16 (a long tool chain ran out halfway), requests maximum reasoning depth from models that expose it, and adds a system-prompt instruction to work in verified steps rather than answer fast.

The model switch is **announced in the chat**, and if the new one is paid it says so: switching silently to a metered service would spend the user's money without telling them. For the same reason it is a switch and not the default — it costs more quota, and on a paid service it costs money.

Saying what is happening. Several seconds can pass between sending and the first word of the answer: the model reasons, calls tools, waits on the network. Before, the user saw a mute pause and concluded it had frozen.

The header now carries an atomic core whose orbitals change speed and colour with state — slow and cyan at rest, fast while reasoning, amber while a tool runs — alongside a line saying the same thing in words, for anyone not watching the animation. The states hook into callbacks the agentic loop already emitted: no mechanism was added, only made visible.

Three layout defects fixed in the same pass, all found by measuring rather than looking:

- The input field shared its row with four buttons and stayed 156 px wide: the

placeholder wrapped and was **clipped**. The text now has its own row with the controls below it. The placeholder length was chosen by testing at 360 px width, not by eye.

- The title "Spectra — il copilota AI di BioSpecInfo" did not fit and truncated

mid-word. Only the name remains; the rest became the status line, which now has a purpose.

- The core's orbitals, when rotating, escaped their box and overlapped the name.

2.9 Using what the provider actually offers

Three things the Anthropic API provides that were not being used. Checked against the official reference rather than recalled — two of the three carry a constraint that would not have been guessed.

Prompt caching. The fixed cost of every request is about 8,150 tokens: 2,000 of system prompt plus 6,128 of tool definitions, identical every turn. Marked for caching they are paid once and then read back at a fraction of the price.

The constraint is that caching works by *prefix*: one differing byte at the start invalidates all of it. But Spectra's prompt also carries user memory and question context, which change every message. Concatenated into one string the cache **would never have hit**. The prompt now travels in two blocks — stable and marked, volatile and not — and the marker also goes on the last tool, because composition order is `tools` → `system` → `messages`.

The Python sandbox, and why it excludes web search. `code_execution` gives the model an environment with sympy, numpy, scipy and matplotlib: numerical integration, symbolic algebra, matrix diagonalisation — things the 32 solvers do not cover because they cannot all be anticipated.

But the new-generation web search **already carries an execution environment** (it filters results by writing code), and declaring `code_execution` as well would create a second one, confusing the model. They are alternatives, not complements. The choice follows Core Mode: normally web search and page fetching, in Core Mode the sandbox — because when maximum power is requested the problem is computational, not documentary. A test verifies the two never appear together.

Page fetching. `web_fetch` joins search: if the user pastes an article link, Spectra reads it instead of merely searching for its title.

Sandbox results arrive as `bash_code_execution_tool_result` and `text_editor_code_execution_tool_result` — not a generic `code_execution_tool_result` — and must be kept in the assistant turn like the web-search blocks, or a server-paused turn cannot be resumed.

---

3. The 51 tools

The list is the one **measured** in `window.BSI_AI_TOOLS` on the running application, not transcribed from memory: this document declared 32 when there were already 35, and three — `analizza_molecola`, `disegna_molecola`, `mostra_spettri` — did not appear at all.

Area	Tools
General computation	`calcola`, `risolvi_equazione`, `analisi_dati`
General chemistry	`bilancia_equazione`, `stechiometria`, `massa_molecolare`, `converti_unita`, `costante_fisica`
Physical chemistry	`termodinamica`, `equilibrio_acido_base`, `cinetica`, `gas_e_soluzioni`, `elettrochimica`
Structure & spectra	`spettroscopia`, `quantistica_e_spettroscopia`, `cristallografia`, `analizza_molecola`, `disegna_molecola`, `mostra_spettri`
Life sciences	`biochimica`, `farmacocinetica`, `valuta_druglikeness`
Physics	`astrofisica`, `nucleare`, `statistica_inferenziale`
External databases	`cerca_pubchem` (NIH), `cerca_letteratura` (PubMed)
Internal data	`cerca_nel_database` (9 datasets), `cerca_molecola`
Macro-areas	`risolvi_matematica`, `risolvi_fisica`, `risolvi_chimica` — the same engine as the three areas' assistant: a natural-language question, or an explicit model and data; working, data with their origin, inverse problem
App control	`naviga_sezione` (194 sections), `apri_strumento` (12 labs), `stato_app`
Memory	`ricorda`, `ricordi`
Animations	`apri_animazione` (6 reaction mechanisms)
Screen and page (Spectra Live)	`leggi_schermo` (subject, section, tab, controls with their values, buttons, note text), `imposta_controllo`, `scegli_scheda`, `premi_pulsante`, `cambia_materia`
Study (Spectra Live)	`consulta_kit` (summaries, formula sheet, glossary, quiz of the subject), `stato_studio` (flashcards due, quiz mistakes), `disegna_grafico`, `apri_strumento_rapido`

The nine tools in the last two rows are defined in `bsi-spectra-live.js` and added to the same `window.BSI_AI_TOOLS` registry when the page starts: the agentic loop treats them like the others, both in the chat and in the console.

> **Web search is not among these thirty-five**, and the distinction matters. The thirty-five are executed **by the application**, in the user's browser. On providers that offer them, the agent adds two tools to the schema that are executed **by the provider** — `web_search` and `web_fetch`, with a per-turn usage cap. Counting them with the others would make them look always available, whereas they depend on the provider chosen.

3.1 Internal datasets exposed

297 synthesis reactions · 118 elements · 67 amino acids · 263 drugs · 63 pathologies · 46 retrosynthetic strategies · 36 drug interactions · 29 metabolic pathways · 29 redox potentials.

Drugs and strategies are the two figures that `tools/verifica-affermazioni.js` measures against the running application on every battery run: here they had been left at 143 and 39 while the app counted 178 and 46.

These are the datasets curated by the author for the application: the agent treats them as a primary source and is instructed to **report discrepancies** between the database and its own memory rather than silently smoothing them over.

---

4. Notable engineering decisions

This section documents the non-obvious choices and the reasoning behind them.

4.1 Expression engine without `eval()`

Problem. Serving arbitrary physical-chemistry computations requires an expression evaluator. `eval()` would solve it in a few lines.

Why it was rejected. The agent reads untrusted external content (PubChem results, PubMed abstracts, web pages). An injection in that content could induce the model to emit a hostile expression; with `eval()` that string would execute in the page context, **where the user's API key lives**.

Solution. Recursive-descent parser with a *closed* set of 27 functions and constants. It has no access to global scope: it can only do arithmetic. Validated against 10 escape attempts (`window.localStorage`, `constructor`, `this`, `fetch(...)`, calls to non-whitelisted functions) — all rejected, including when executed in a real browser.

4.2 Preserving reasoning blocks

On models with adaptive reasoning, `thinking` blocks are part of the assistant turn and must be **echoed back unchanged, cryptographic signature included**, on the next tool-use round. Dropping them causes the request to be rejected.

The implementation accumulates `thinking_delta` and `signature_delta` from the stream and reinserts them **first** in the reconstructed turn. Verified in integration tests: signature preserved, ordering correct.

4.3 Resuming paused turns (`pause_turn`)

Web search is a server-side tool. When its internal loop exhausts its iterations, the response ends with `stop_reason: "pause_turn"`.

The turn must be resumed by **sending the assistant turn back as-is, without adding any user message**: the server recognises the trailing `server_tool_use` block and continues from there. Appending a "continue" message would break the mechanism.

4.4 Per-model configuration, not global

The four Claude tiers do not accept the same parameters:

Model	`effort`	Web search	Notes
Fable 5.1	`xhigh`	yes (8)	refusal fallbacks; reasoning always on
Opus 5	`high`	yes (6)	default
Sonnet 5	`high`	yes (5)	
Haiku 4.5	**unsupported**	**unsupported**	sending them returns HTTP 400

On all recent models `temperature`, `top_p` and `budget_tokens` were removed from the API and their presence causes an error: none of the four sends them. Every parameter is therefore conditioned on the provider rather than set globally.

4.5 Multimodal input and material reading

The agent accepts images, PDFs and text files, several at once, treated as consecutive pages of a single document. Three constraints shaped the implementation:

- **Images are downscaled to 2000 px** and re-encoded as JPEG before sending.

A phone photo drops from roughly 1 MB to 440 KB: without this step a single image would approach the request ceiling and multiply token cost. 2000 px is the minimum for reading small handwriting — at 1600 px fine script became illegible.

- **The PDF page ceiling depends on the model**, it is not a constant: 600

pages for models with a 1M-token window, 100 for 200K ones. Page counting is done by reading `/Type /Page` objects from the file bytes, without libraries.

- **History stores a 160 px thumbnail**, not the image that was sent. Storing

the large one (440 KB each) filled `localStorage` within a dozen photos, and once the quota is exceeded every subsequent write fails silently.

Six quick actions turn the material into a study artefact (summary, concept map, outline, flashcards, exam questions, transcription) and two translate it. For transcriptions the agent is instructed to write `[illegible]` rather than guess: an invented chemical term is worse

than a declared gap.

4.6 In-house graph renderer, no CDN

Concept maps are produced by the model as Mermaid diagrams and drawn by a renderer written into the application (~190 lines): longest-path layering from the roots with cycle breaking, barycentre reordering over three passes to reduce edge crossings, SVG output with Bézier curves.

Not loading Mermaid from a CDN follows from the app's offline-first nature: a map must be drawable without a network. Unsupported syntaxes are recognised and left as a readable code block instead of being drawn badly.

4.7 Voice command with wake word

User-activated continuous listening: the phrase "Hey Spectra" followed by a command sends it automatically. Two non-obvious details:

- the browser's speech recognition **stops on its own** after a few seconds of

silence and must be restarted, otherwise listening dies at the first pause;

- **all transcription alternatives** are examined, not just the first, and the

most common phonetic variants are accepted: Italian recognition renders "Spectra" in many different ways.

Denying microphone permission stops the loop instead of retrying forever. The word "spettroscopia", extremely frequent in this domain, was verified as non-triggering.

4.8 Guard against non-finite results

A systematic audit revealed that, given legitimate degenerate inputs (zero wavelength, zero volume, zero half-life, a transition between identical levels), eight solvers returned `ok: true` with `Infinity` or `NaN` among their fields.

This is the most insidious failure in this context: an exception is noticed, a formally valid but meaningless value is reported to the user as correct. The fix is a single check at the point where all results pass through: if a numeric field is not finite, the result becomes an explicit error naming the fields and stating the probable cause. It also covers tools added in the future.

4.9 Graceful degradation

- **Models without reliable function calling** (some free tiers): on the first

tool-related error the request is retried once in text-only mode instead of failing.

- **Idle timeout (45 s)**, reset on every byte received: a long answer is never

truncated, but a dead connection is reported immediately with an explicit message.

- `localStorage` **quota**: history is capped (30 conversations, 100 messages)

with progressive pruning. Without this cap, exceeding the quota made every subsequent write fail *silently*, **including saving the API key**.

- **Classifier refusals** (`stop_reason: "refusal"`, HTTP 200): detected and

surfaced, instead of appearing as an empty answer.

4.10 Spectra Live: voice, screen and local commands

`bsi-spectra-live.js` adds a full-screen console. It opens from any screen with Spectra's only button: the face in a circle (`#bsiJV-fab`), which since v239 takes the place of the «Spectra» pill (`#bsi-spectra-fab`). The pill stays in the document, hidden by the `bsiJV-solo-volto` class on `body`, because other modules and the benches use it to open the chat; from the console the chat opens with 💬. The circle appears at once with the vector face and switches to the portrait (`models/spectra-ritratto.jpg`) after the `load` event. The «◈ Live» button in Spectra's header and `Ctrl+Shift+J` remain:

- **hands-free conversation**: the browser's speech recognition listens, the

answer is spoken sentence by sentence with the system's speech synthesis (a male voice is preferred, rate 0.95, pitch 0.7) or, since v239 and only with a saved OpenAI key, with the voice of `bsi-spectra-voce.js` (§4.15); the console then listens again; «stop» or a tap interrupts it. If either capability is missing in the browser, the console says so;

- **local commands**, recognised with regular expressions before the model is

called and therefore available without network and without a key: open a section or switch subject, compute, differentiate and integrate (with the app's symbolic engine), convert units, look up a constant, define a term, plot a graph, quiz, review, move a slider, change tab, pause and resume an animation, tell the time and date, and the mind's commands («what do you remember», «how are you», «forget everything»);

- **screen context**: every question that goes to the model carries a short

summary of what the user is looking at (subject, section, tab, control values, up to 700 characters of visible text). If the question contains words such as «this», «here», «chart», or the user pressed 👁, an image of the tab's visible canvases is attached as well.

Everything that goes to the model takes the same path as the chat, to the provider the user chose. Speech recognition is the browser's: where the browser runs it on a remote service, the audio follows that browser's rules, not BioSpecInfo's.

Layout and navigation (v238). On a phone with the app installed, Android's status bar and keys sit over the page. The console uses `env(safe-area-inset-top/bottom)` when the system reports them and, with the class `.bsiJV-bordi` (full-screen window on a touch device, or installed app), at least 34 px at the top and 46 px at the bottom. Positions are no longer fixed: on every frame the header, telemetry and control bar are measured, and the face, state, thought and dialogue take the remaining space. The 🧭 navigator builds from the bar itself, for each of the three subjects, the list of groups and sections, then the quick tools

(`BSIAttrezzi`) and the locally recognised phrases; the 💬 button switches to the chat; the context shortcuts change with the open section. The telemetry shows USER (guest or creator), LEVEL and FOCUS.

Level and concentration. `difficolta(testo)` starts from the level of the open section, i.e. the first level of the `bsi-percorsi.js` path in which it appears (high school 0, bachelor 1, master 2, PhD 3: value 0.15 + 0.22·level), adds 0.3 for advanced-course words (theorem, Hamiltonian, eigenvalues, …), up to 0.2 for symbols, 0.1 for a long question, subtracts 0.25 for «in simple words», and clamps the result between 0.05 and 1. The value goes to the face through `concentra()`.

Initiative. With the console open, waiting and silent for more than a minute, if the mind has a thought of weight ≥ 2 (a break, mistakes to review, the late hour) Spectra says it, at most once every three minutes and never the same twice, with the matching expression.

4.11 The face

`bsi-spectra-androide.js` shows the «Noir» 3D model by Skyfox13 in the console (Sketchfab, CC BY 4.0, modified; attribution in the console and in `07-SBOM.md`). The model has neither a skeleton nor mouth shapes, so the animation is built in code:

- the **head** is detached from the neck and turns with the gaze and the

breathing;

- the **mouth** opens on the spoken text: the jaw displacement is computed

once per vertex and the shader multiplies it by the current opening;

- the **expressions** are 17 emotions, each a blend of **11 expressive

fields **(smile, rounded lips, wide mouth, raised and furrowed brows, …). WebGL allows no more than 16 attributes per vertex, so the eleven fields are kept in memory and** blended on the CPU** into a single attribute when the expression changes, recomputing only the vertices some field moves;

- the emotion of a sentence comes from Italian and English keywords

(`emozioneDi`); with no signal it is «serena» (calm);

- the **eyes** (version 3.0, v238) have **real eyelids**: the vertices of

each eye are deformed on the CPU around its centre, with the opening and asymmetry of each emotion (joy narrows them, surprise widens them, doubt makes them asymmetric); the gaze moves them; the **tint** changes with the emotion within the theme's blue family (`#1a3050` at rest, brighter in joy and surprise);

- **concentration** (0–1, from `difficolta()`) half closes the eyes, brightens

them, draws the brows together, presses the lips and slows the blinking;

- every facial muscle is a **damped spring**, so expressions have inertia and

a small settling; every few seconds a quarter-second **micro-expression** appears; the mouth uses **coarticulation**: the next letter's viseme starts before the current one ends;

- neutral lights (the saturated blue back-lights tinted the metal), the

screen's real density up to 3× and anisotropic filtering on the metal maps;

- after **two minutes without contact** (touch, voice, answer) the face falls

asleep: heavy eyelids, lowered head, slow breathing; the first touch wakes it with surprise and then a smile. The gaze follows the pointer and, on a phone, the touched point.

Like the mind, it is a **simulation of behaviour, not consciousness**.

Without WebGL, or if the model does not arrive, the console uses the head built by `bsi-spectra-robot.js` and, as a last resort, the vector portrait of `bsi-spectra-volto.js`. With «reduce motion» the head stays still.

4.12 The mind

`bsi-spectra-mente.js` gives Spectra continuity across sessions. **It is a simulation of behaviour, not consciousness**: five numbers between 0 and 1 and a few rules, not subjective states.

- **Memory**: conversation episodes (at most 300, each with time, subject,

section and the first 160 characters of the question), the time spent in each section, the study hours, the emotions detected in the user's sentences (at most 60).

- **Internal state**: curiosity (rises with new questions), attention (rises

when working together, falls during pauses), care (grows with session length and at night), mood (moves with the tone of the user's sentences and drifts back to an equilibrium), bond (grows with every conversation, ever more slowly). The state picks the face's background emotion and the tone of the greeting.

- **Thoughts**: suggestions derived from these data (a break after 90

minutes, reviewing quiz mistakes, a subject left behind).

- **Summary for the model**: every question that goes to the model carries a

summary of study time, frequent sections, recent topics and internal state.

Everything is saved only in the browser's `localStorage`, key `bsi-spectra-mente-v1` (`11-Data-Model.md`); «what do you remember» shows it, «forget everything» erases it.

4.13 The creator

`bsi-spectra-creatore.js` (v238) tells Spectra who created it: Samuele Pio Provenzano. «Who created you?» has a local answer. «I am your creator» opens a proof: the console field becomes a password field for a minute, voice listening stops, and the text sent **does not go through** `elabora()`: no dialogue, memory, voice or model. Its SHA-256 fingerprint (`crypto.subtle`) is computed and compared with the File Manager's; the phrase does not appear in the code. Three wrong attempts lead to a one-minute wait.

Once recognised, Spectra writes only `bsi-spectra-creatore-v1` (`ok`, `dal`, `ultima`) to `localStorage`, greets the author by name when the console opens, shows «CREATOR» in the telemetry and adds to the model's context (`contesto()`, also inside the mind's `coscienza()`) that it is talking with its author. For a guest the context says the user has not been recognised and that the phrase must never be asked for in chat. «Forget that I am your creator» removes the mark; so does «Clear data → Persistent memory».

Limitation. Like the File Manager password it is a deterrent, not an access control: the fingerprint is in the code and the mark can be written by hand by whoever holds the device (`09-Release-Conformance-Statement.md`, D-13).

4.14 The guided lesson

`bsi-lezione.js` (v239, object `BSILezione`) turns every section into a lesson in chapters led by Spectra. It does not use the model: the lesson is built by reading the open section. Only the voice goes online, and only with a saved OpenAI key (§4.15).

- **Construction** (`costruisci(section)`): an opening step with title and

subtitle; one chapter per tab (`.bsiFX-schede button`), whose steps are read when the tab is open: the explanation split into pieces of one or two sentences (at most three pieces, about 260 characters each), the first visible canvas or 3D scene, up to four sliders and selects; for sections without tabs a single chapter with the visible headings and paragraphs and the same controls. Then a **check question** from the subject's study kit (`BSIStudio.dati(subject).quiz`): the one with the most words (longer than four letters) in common with title, subtitle and tabs wins, with double weight for the words of its topic; a score of at least 2 is required, otherwise there is no check. Finally the **summary** of the chapters.

- **Demonstrations**: a slider goes from its current value to the minimum, the

maximum and back to the starting value in about 4 s, firing `input` and `change` events as the user would; a select goes through its first five options and returns to the starting one. Skipping ahead half-way, or closing the lesson, puts the control back as it was.

- **Player**: panel `#bsiLZ` (on the right on a large screen, at the bottom

below 760 px; from 1100 px `#main` shifts left), progress by chapter, text appearing word by word, ◀ ▶, pause, voice, `Esc` to close, arrow keys to move. The spotlight `#bsiLZ-luce` dims the rest and follows the piece being discussed. The voice is Spectra Live's (`BSISpectraLive.parla`) and, if the console is not there, `BSISpectraVoce.parla`; without voice, reading time depends on the length of the text. With «reduce motion» sliders jump to the value, text appears all at once and the spotlight does not pulse.

- **Floating buttons**: they are hidden during the lesson (including those

with `visibility` written inline, whose value is noted and restored on closing).

- **Start**: the «🎓 Guided lesson with Spectra» button the module adds under

the title of every visible section; Spectra Live's local command «give me a lesson» («fammi lezione»), which closes the console and starts the lesson of the open section; the 🎓 shortcut in the console.

The lesson has no keys of its own in `localStorage`; at the end it signals an emotion to Spectra's mind (`sente('fiera', 'lezione')`), which the mind records in `bsi-spectra-mente-v1` like the others.

4.15 Embodied Spectra: the studio, the body, the voice

Since v239 Spectra also has a body, on a page of its own: `spectra-studio.html`, reached from the console's 🧭 navigator. The full design (model requirements, choice of stack, modules, state, lesson cycle, migration, domain coverage) is in the Italian design documents in `docs/spectra/`: analysis and plan, architecture, domain map. What is in the code is summarised here.

- **The studio** (`spectra-studio.html`): a three.js r128 scene (grid floor,

neon lights, holographic bench) with Spectra standing; an accessible panel with all the text. The lessons in `bsi-spectra-studio-lezioni.js` (`BSIStudioLezioni`: `LEZIONI`, `SCENE`, `calcolo`) follow four phases: **predict** (choices with the most common misconceptions and one right answer, each explained), **test** (the simulation with the numbers measured in the scene), **prove** (the derivation step by step), **reveal** (why, connections, exercises on four levels). The computations live in `calcolo` and also run in node: fall with semi-implicit Euler (and optional quadratic drag), VSEPR as the minimum of the energy w_i·w_j/r of domains on the sphere (w = 1 for bonds, 1.25 for lone pairs), partial sums, tail, Fourier series of x^2, Monte Carlo coprimes.

- **The studio's memory**: `bsi-spectra-studio-v1` keeps, per lesson, the

predictions (`quando`, `scelta`, `giusta`) and the exercises opened; the recommended level is the number of right predictions plus exercises opened, up to 3. Each prediction also goes to the mind (`BSISpectraMente.ricorda`). The 🔊/🔇 switch in the action bar turns the studio's voice off and the choice is kept in `bsi-spectra-studio-voce` (`'0'` = off). Both keys are covered by «Clear data → Persistent memory».

- **The body** (`bsi-spectra-corpo.js`, `BSISpectraCorpo`): the driver

recognises bones, ARKit blendshapes, hair and clips of a humanoid GLB; every pose is a direction in the world, converted into the bone's local frame (`orienta`, a minimal inverse kinematics valid for any rig); emotions, lip sync, eyelids, gaze, gestures, walking and sitting are blended with springs. The mannequin from `BSISpectraCorpo.manichino()` (bones with Mixamo names in T-pose, 26 ARKit blendshapes, red hair that the driver turns black) is used **only by the driver's bench** (`opz.manichino === true`): since v240 it is no longer Spectra anywhere. **Spectra's body** is chosen by the shared controller `BSISpectraFigura.Spectra()` (`bsi-spectra-figura.js`), in the studio and in the AI sections. Since v241 the body is **always present**: `carica()` downloads `models/spectra-corpo.bsi`, the «Bounty Hunter» (CGTrader, Royalty Free licence) embedded as a block packed by `tools/impacchetta-corpo.js`, and reopens it with `apri`; the published file is not a reusable

`.glb` (it is not absolute protection). While it arrives, «Spectra sta arrivando...»; if it does not, «↻ Riprova». Only in the studio, in development, if the block is missing `models/spectra-corpo.glb` (out of git) is tried; in the app (`soloDispositivo`) no development file is requested, no 404s. The « 🧍 Corpo» button, the user-loaded body in IndexedDB and the `models/spectra-cyberpunk.glb` fallback were removed; an old body in `bsi-spectra-modelli` is deleted on start. No substitute. A body without a skeleton goes through the automatic rig (`bsi-spectra-rig.js`); the «Noir» face (`BSISpectraAndroide.crea(cont, { scena, renderer, senzaBusto: true })`) goes on the collar, aligned by measuring the chin, with only the face visible, «solidale» with the body's head.

- **The figure in the AI sections** (`BSISpectraFigura.monta`, `sezioni`,

v240): one figure, moved into whichever section is open among Spectra AI and the three areas with the assistant, shows Spectra standing in a lab-style frame and **reacts to the app's events**: `bsi-spectra-stato` (thinking, tools, writing) → concentrated, hand on chin, «SPECTRA · PENSA»; `bsi-spectra-risposta` (the model's answer) → explains with the hands, lip sync of body and face from the text, emotion from the content, «SPECTRA · PARLA»; an error → doubt; `bsi-spectra-dice`, dispatched by Spectra Live's `parla()` and by the local assistant (`bsi-assistente.js`) when it solves an exercise («Ecco lo svolgimento: ...», or doubt if it did not understand) → the figure says that text. It renders only when visible and the page is in the foreground. The events are one-way: the figure does not change what the model answers.

- **The figure in the Spectra Live console** (v242): with the console open,

`sezioni()` moves the shared figure into `#bsiJV-figura` (between the telemetry and the dialogue; on a large screen on the right), adds the `con-figura` class and hides the bust `#bsiJV-volto`: instead of the face alone, Spectra with the body and the hologram. The console emits `bsi-spectra-stato` at every state change (waiting, listening, thinking, speaking): thinking → concentrated, hand on chin; listening → curious, «SPECTRA · LISTENING TO YOU». The console's event carries the `daLive: true` signature and the console ignores its own events: without the signature the console, which listens to the same event for the other modules, called itself forever (a recursion found by the bench). With the console closed, the figure returns to the section.

- «🧠 **Spiega**» (`bsi-spectra-spiega.js`, `BSISpiega`, v240): the second

tab of the Spectra window, after «💬 Chat», for an **exhaustive explanation** of any chemistry, physics or mathematics question. The system prompt (`sistema(level, subject)`) sets the level — five, from High school to Advanced PhD · frontier, each with its instructions in `PROFONDITA` and «do not simplify below this level and do not leave out anything relevant» — and a **mandatory structure** with `##` titles: in short; what you are looking at (only with the screen: quantitative reading of axes, units, curves and values; for NMR spectra δ, multiplicity, J, integral and assignment of every signal, IR bands and groups, MS molecular ion, fragments and isotopes, UV-Vis λmax, transitions and ε, then the structure and its consistency; for 3D scenes geometry, symmetry, nodes, critical points, isosurfaces and fields; for mathematical graphs domain, asymptotes, zeros, extrema, inflections, convergence); foundations; numbered derivation; computed examples; limiting cases and common errors; links be-

tween the three subjects; applications and frontier (literature only if certain); three self-check questions with answers. **The screen** (`schermo()`, checkbox «📺 Includi lo schermo»): section, tab, every control with value and range, visible tables (up to 4 × 40 rows), visible text, number of graphs; `immagine()` composes the visible canvases (`BSILab3D` scenes re-rendered first, via `BSISpectraLive.cattura`) and the visible SVGs rasterised, at most 1400 px wide, as JPEG. The image goes to models that accept images, with the chat's per-provider conversion, in the first turn only; follow-up questions continue the conversation, which is kept in memory for the session. Provider and key are the chat's (`window.bsiLLM`). The shared figure moves into the tab (priority over the sections), thinks while the model writes, then speaks and gestures. **Without a model it does not pretend**: it says so and shows what the app knows on its own (the local assistant's recognition, the chapters of the section's guided lesson, the screen data as text). With the screen included, text and image go to the configured provider together with the question. **The catalogue** (`catalogo()`): every section button of the bar (`.nav-group-panel [data-s]`) with its group label and subject (`BSIMaterie.materiaDi`), 201 sections since v242 (chemistry 78, physics 63, mathematics 31, shared 29), read from the bar rather than written by hand. The «🗂️ Tutte le sezioni nell'ologramma» card lists them by group, with search and a subject filter; `mostraSezione(id)` opens the section behind the window (`BSISpectraLive.calcolo.apriSezione`, which switches subject if needed), the hologram shows it live and the screen checkbox is checked, unless the user unchecked it; `giro(materia)` («🎞️ Mostrami tutto») shows them in turn every 7 s, with the counter «n/N · name», and `giroFerma()` stops it. `sezioneDi(testo)` finds the section a question names (meaningful words of the names, also by stem; acronyms MS, IR, UV, NMR, MO… weighted more; synonyms «spettro di massa» → MS, «infrarosso» → IR, «risonanza magnetica» → NMR, «orbitali molecolari» → MO; ties go to the shorter name; none if the question names no topic): asking without the screen included, the named section opens, is shown in the hologram and the explanation includes its screen. **A box unchecked by the user is respected** (`ST.schermoNo`, set by the checkbox's `change` event): the section still opens and is shown, but the model receives only the question, with no screen text or image; the quick prompts about the screen check it again (explicit choice). A section without graphs is projected on the hologram as a card (title, tab, up to 6 controls with values, visible text), so every section is showable.

- **The interactive hologram, «⛶ Interagisci»** (`bsi-spectra-olointerattivo.js`, `BSIOloInterattivo`, v242): the button on the shared figure's hologram opens `#bsi-olo-ov`, full screen; `sezioni()` gives precedence to `#bsiOI-figura` while the window is open, so Spectra with the body moves in, and with the window closed returns to the section. The mode comes from the hologram's scene (planets → universe; spectrum and molecule → spectra; surface and orbital → mathematics; wave → physics; the «screen» according to the open section). **No model call**: spectra come from the six reference molecules (typical literature values) or from `leggiTraccia` (candidate assignments from group-frequency, chemical-shift and neutral-loss tables, taking band shape into account); planets from the NASA fact sheet with Kepler and vis-viva computed; mathematics and physics from the local assistant (`bsi-assistente.js`, `bsi-mat-cas.js`), whose working becomes the list of steps and whose result drives the graph (function and derivative with zeros and stationary points, pro-

jectile trajectory, pendulum). Each step has a title and a text; «🔊 Spectra spiega i passaggi» hands them one by one to `BSISpectraLive.parla` and the figure says them with lip-sync and gestures; tapping a peak or a planet calls `scegli(i)`. Only **«🧠 Approfondisci»** goes to a model, through «🧠 Spiega» with the hologram's data, and under its rules.

- **«⚡ Explore with Spectra»** (`bsi-spectra-esplora.js`, `BSIEsplora`,

v242): three sections, one per subject (`sesplchim`, `sesplfis`, `sesplmat`), registered with `BSIFisica.sezione`, in the style of explorable lessons. Ten chapters per subject (`CAPITOLI`, levels 0→4, from high school to advanced PhD · frontier), each with the phenomenon, a four-answer prediction with the reason for each, in 19 chapters a computed slider (`calcola(id, v)`), the linked app section and the hologram scene. The shared figure enters the section with the `bsiEX-palco` class (`position: sticky` under the measured app header; shorter on phones) and stays on stage while scrolling; the chapter crossing the middle of the free space under the figure becomes active: `bsi-spectra-dice` (Spectra tells it, curious) and `olo.mostra(scena)`. The prediction emits `bsi-spectra-dice` with joy or surprise. «📺 Guardalo dal vivo» opens the window on « 🧠 Spiega» and the section behind it (`BSISpiega.mostraSezione`); «🧠 Spiegamelo tutto» calls `BSISpiega.chiedi(title + question, { schermo: true, livello })`, so it goes through the same provider and the same screen rules as «🧠 Spiega»; «🔊 Ascolta Spectra» uses `BSISpectraLive.parla`. The Explore sections count as AI sections: closing the window returns there. No model call without a user gesture; predictions stay in `bsi-esplora-v1`.

- **The voice** (`bsi-spectra-voce.js`, `BSISpectraVoce`): `qualita()` is

`'audio'` with an OpenAI key in `bsi_api_keys` and Web Audio available, `'sistema'` with the browser's speech synthesis only, `'testo'` without. In audio mode the text (up to 3,800 characters) goes to `api.openai.com/v1/audio/speech` (`gpt-4o-mini-tts`, voice «onyx», with instructions on the timbre; `tts-1` if that fails); the decoded audio stays in an in-memory cache per sentence and goes through the `TIMBRO` chain (bass +5 dB below 140 Hz, −3 dB at 350 Hz, +3 dB at 2.8 kHz, ring modulator at 55 Hz at 12 %, 4.3 ms comb, saturation, synthetic 1.4 s reverb at 16 %, compressor). An analyser gives energy and spectral centroid every frame for the lip sync. The text to be spoken leaves the device: see `03-Security-Privacy-Compliance.md` §1.2.

Like the mind and the face, the body is a **simulation of behaviour, not consciousness**.

---

5. Verification

Verification was carried out at three levels, with recorded outcomes.

5.1 Numeric correctness — against reference values

Area	Case	Expected	Obtained
Balancing	$KMnO_4$ + HCl → …	2:16:2:2:8:5	✓ exact
Molecular mass	$K_4[Fe(CN)_6]$ (nested)	368.35	368.345

Area	Case	Expected	Obtained
Molecular mass	$CuSO_4 \cdot 5H_2O$ (hydrate)	249.68	249.677
Equilibrium	acetic acid 0.1 M, K_a $1.8 \cdot 10^{-5}$	pH 2.874	2.875
Thermodynamics	ΔG (N_2O_4/NO_2) at 298 K	+4.79 kJ/mol	✓
Kinetics	E_a from k(300 K), k(320 K)	55.3 kJ/mol	55.33
Real gases	van der Waals CO_2	−10 % deviation	−10.13 %
Spectroscopy	bromine M+2	97.3 %	✓
Biochemistry	glycylglycine	132.12 Da	✓
Pharmacokinetics	t½ from V_d 50 L, Cl 5 L/h	6.93 h	✓
Astrophysics	solar emission peak (Wien)	501.5 nm	501.52
Astrophysics	Earth escape velocity	11.19 km/s	11.186
Nuclear	^{56}Fe binding energy	8.79 MeV/nucleon	8.790
Statistics	critical t, 95 %, df = 4	2.7764	✓
Crystallography	copper density (fcc)	8.96 g/cm³	8.935

P-values are not tabulated: they are computed with the regularised incomplete beta function (Lentz continued fraction) and the incomplete gamma function.

5.2 Robustness — cases that must fail

Verified as **rejected**: unbalanceable equations, formulas with unbalanced parentheses or non-existent elements, species absent from the equation, invalid database types, and the 10 injection attempts against the expression engine.

5.3 Integration — real browser

Automated tests with headless Chromium across all 14 application pages: no JavaScript errors, no missing resources. All 194 sections of `index.html` and the 18 tabs of the Astrochemistry module were traversed one by one. Request bodies were inspected for the four Claude tiers, and a full cycle with web search, turn suspension and resumption was simulated.

---

6. Quantitative summary

Metric	Value
Component size	6,254 lines
Tools	51 (38 in `bsi-ai-hub.js`, 9 in `bsi-spectra-live.js`, 3 in `bsi-spectra-agenda.js`, 1 in `bsi-spectra-risveglio.js`)
Model configurations	10 (3 free)
Scientific areas covered	13

Metric	Value
Records in exposed internal datasets	over 800
Max agentic loop rounds	10
History sent to the model	40 turns
Runtime dependencies	none

---

7. Stated limitations

Documented for technical honesty; these are deliberate choices, not omissions.

- **The API key lives in the browser** — unless the proxy is deployed. In

the default configuration the key never leaves the device, but it is accessible to anyone with access to that browser, and every user must obtain their own. The Worker in section 2.5 removes this limitation by moving keys to a server, at the cost of one component to maintain outside the page: a distribution choice, not a requirement.

- **Network tools depend on service availability.** PubChem and PubMed are

queried directly from the browser; when unreachable, the tool returns an explicit error and the agent is instructed to say so rather than substituting an estimate.

- **Solvers apply simplified models** where the literature offers more refined

ones (e.g. Woodward–Fieser for UV, or ATP yields using average P/O ratios). Each tool states the method used in its own result.

- **The screen context and the mind's summary go to the provider.** When a

question goes to the model, the chosen provider also receives the summary of the open section and, if there is memory, the mind's summary (§4.10, §4.12). Only the answers to local commands stay in the browser.

- **Verification is functional and numeric**, not formal: there is no proof of

solver correctness, but a set of reference cases drawn from the teaching literature.

Companion document to the BioSpecInfo technical dossier. See also: `01-Software-Architecture-Document.md`, `02-Verification-Validation-Report.md`, `03-Security-Privacy-Compliance.md`.